FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *Le Realtime de Supabase est puissant mais traître. Une mauvaise dépendance dans un useEffect, un setLoading mal placé, un userId capturé en closure — et la magie devient cauchemar. Ces règles sont le fruit de trois audits de code successifs.*

LE BUG LE PLUS COURANT

Règle 1 — Anti-loop subscription

✕ ANTI-PATTERN

CODE TYPESCRIPT

```typescript
// MAUVAIS : setLoading(true) dans le callback
channel.on("postgres_changes", ..., () => {
  setLoading(true);
  await fetchData();
  setLoading(false);
});

// → Re-render à chaque event
// → useEffect se ré-exécute
// → Nouvelle subscription créée
// → Boucle visible (spinner clignote)
```

✅ **PATTERN CORRECT**

CODE **TYPESCRIPT**

```typescript
async function fetchData(showLoading = false) {
  if (isFetchingRef.current) return;
  isFetchingRef.current = true;
  if (showLoading) setLoading(true);
  try { /* ... */ } finally {
    isFetchingRef.current = false;
    if (showLoading) setLoading(false);
  }
}

// Initial mount ⟶ spinner OK
fetchData(true);

// Realtime callback ⟶ silent refetch
channel.on("postgres_changes", ..., () ⇒ fetchData(false));
```

VISIBILITÉ

Règle 2 — Toujours un handler d'erreur subscribe

Sans handler d'erreur, une déconnexion WebSocket (mode veille, changement Wi-Fi) est **invisible**. L'utilisateur croit que le chat fonctionne, mais aucun nouveau message n'arrive jamais.

CODE TYPESCRIPT

```typescript
channel.subscribe((status, err) => {
  if (status === "CHANNEL_ERROR" || status === "CLOSED" || err) {
    console.error("[Realtime] Channel error:", { status, err });
    // 1. Marquer le state comme "déconnecté"
    setRealtimeStatus("disconnected");
    // 2. Tenter une reconnexion après backoff
    setTimeout(() => recreateChannel(), 2000);
  }
  if (status === "SUBSCRIBED") {
    setRealtimeStatus("connected");
  }
});
```

MEMORY LEAKS

Règle 3 — Garde isMounted avant channel async

Quand un useEffect contient une fonction async qui crée un channel, le composant peut se démonter avant que le channel soit prêt. Sans garde, on crée des channels orphelins qui consomment des ressources.

CODE TYPESCRIPT

```
useEffect(() => {
  let isMounted = true;

  async function setup() {
    const data = await fetchInitial();
    if (!isMounted) return;  // ← critique

    const channel = supabase.channel(`thread:${threadId}`)
      .on("postgres_changes", ... , handleEvent)
      .subscribe();

    channelRef.current = channel;
  }

  setup();

  return () => {
    isMounted = false;
    if (channelRef.current) {
      supabase.removeChannel(channelRef.current);
    }
  };
}, [threadId]);
```

STABLE REFS

Règle 4 — Pas d'arrays d'objets dans les dep arrays

❌ ANTI-PATTERN

CODE TYPESCRIPT

```
useEffect(() => {
  const channel = supabase.channel(...);
  // ...
}, [messages]);  // messages est un array → ref change à chaque render
```

✅ PATTERN CORRECT

CODE TYPESCRIPT

```
// Synchroniser via ref, mais ne pas dépendre dans le useEffect principal
const messagesRef = useRef(messages);
useEffect(() => { messagesRef.current = messages; }, [messages]);

useEffect(() => {
  const channel = supabase.channel(...).on(..., () => {
    // Toujours messagesRef.current, pas messages
    const latest = messagesRef.current;
    // ...
  });
}, []);  // pas de deps fluctuantes
```

RÉFÉRENTIEL

Règle 5 — Stale closures sur userId / props

Une subscription créée au mount capture `userId` en closure. Si l'auth change (rotation, switch d'onglet), le channel garde l'ancien `userId`.

CODE TYPESCRIPT

```
const userIdRef = useRef(userId);
useEffect(() => { userIdRef.current = userId; }, [userId]);

useEffect(() => {
  const channel = supabase.channel( ... ).on( ... , (payload) => {
    if (payload.new.recipient_id === userIdRef.current) {
      // valeur fraîche, toujours à jour
      handleNewMessage(payload.new);
    }
  });
}, []);
```

PERFORMANCE

Règle 6 — Filtrer les events côté client si filtre serveur impossible

Le filtre Supabase Realtime ne supporte que `=` sur une seule colonne. Pour des conditions plus complexes, vérifier dans le callback :

CODE TYPESCRIPT

```
channel.on("postgres_changes",
  { event: "INSERT", schema: "public", table: "chat_messages" },
  (payload) => {
    // Filtrer côté client : seulement nos conversations
    if (!ourConversationIds.has(payload.new.conversation_id)) return;
    // ...
  }
);
```

CHOIX DU CANAL

Règle 7 — broadcast pour l'éphémère, postgres_changes pour le persisté

CAS D'USAGE	MODE	POURQUOI
Nouveau message chat	postgres_changes	Persisté + Realtime
Indicateur de frappe	broadcast	Éphémère, 0 écriture DB
Réaction emoji ajoutée	postgres_changes	Persisté + Realtime
Présence (online/offline)	presence	API dédiée Supabase
Curseur en édition collab	broadcast	Volume très élevé, pas besoin de log

CONFIGURATION

Règle 8 — Activer la publication Realtime sur la table

Beaucoup d'heures perdues à debugger « pourquoi je ne reçois pas les events ». Réponse : la table n'est pas dans la publication Realtime de Postgres.

CODE SQL

```sql
ALTER PUBLICATION supabase_realtime
  ADD TABLE chat_messages, forum_messages, forum_reactions;
```

À VÉRIFIER

Cette config se fait via Supabase Studio ou migration SQL — facile à oublier en local mais critique en prod.

DISCIPLINE

Checklist nouveau channel Realtime

1. Garde `isMounted` avant `supabase.channel`.
2. Refs synchronisées pour toutes les valeurs « volatiles » utilisées dans le callback.
3. Handler d'erreur sur `.subscribe()`.
4. Cleanup avec `removeChannel` dans le return du useEffect.
5. Filtres côté client si nécessaire.
6. Ne JAMAIS appeler `setLoading(true)` dans le callback.
7. Vérifier que la table est dans la publication Realtime.

Sakata.com — Documentation interne · architecture-realtime-strategy · v2026.4

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM